signal and image proccecing lab

DR.nouri

electrical engineering department

Ahmadreza Majlesara 400101861
Parsa Hatami 400100962

Lab 8

December 16, 2024

Theory: Gradient Descent for Deconvolution

To address the problem of deconvolution, we consider the iterative update formula:

$$f_{k+1} = f_k + \beta D^T (g - Df_k), \quad f_0 = 0$$

where β is the learning rate, D represents the convolution operator (e.g., the PSF in the case of Gaussian blur), and g is the observed blurred image. This formula iteratively minimizes the cost function $\|g - Df\|^2$, which measures the difference between the observed blurred image and the reconstructed image.

solution

To derive the update formula, we begin by expanding the expression for the squared error $\|g - Df\|^2$:

$$\|g - Df\|^2 = (g - Df)^T (g - Df) = g^T g - 2(g^T Df) + f^T (D^T D) f.$$

Next, we calculate the gradient of the cost function $\|g - Df\|^2$ with respect to f:

$$\frac{\partial}{\partial f} \|g - Df\|^2 = -2D^T (g - Df).$$

Applying the gradient descent method to minimize this cost function, the iterative update rule for f can be written as:

$$f_{k+1} = f_k + \beta D^T (g - Df_k),$$

where β is the learning rate and f_0 is initialized to zero.
Thus, the gradient descent formula is obtained.

Question 1

Read the first slice of the image `t2.jpg` from the `S2_Q1_utils` directory, and add Gaussian noise with a variance of 15. Then, create a square kernel of size 4×4, centered in the Fourier domain, and apply it to filter the noisy image. Normalize the kernel by ensuring the sum of its elements equals 1. Finally, use the `imgaussfilt` function to filter the noisy image with a Gaussian kernel and compare the results.

solution

we used this code for this part:

```
img = imread("S2_Q1_utils/t2.jpg");
figure;
imshow(img);
title('S2-Q1-utils/t2.jpg', 'Interpreter', 'latex');

img_d = img(:,:,1);

noiseVariance = 0.015;
noiseMean = 0;
img_d_noisy = imnoise(img_d, 'gaussian', noiseMean, noiseVariance);

k_size = 4;
[X, Y] = ndgrid(-size(img,1)/2:size(img,1)/2-1, -size(img,2)/2:size(img,2)/2-1);
kernel = abs(X) < k_size & abs(Y) < k_size;
imshow(kernel);
title(['Binary Kernel, $k$ size = ', num2str(k_size)], 'Interpreter', 'latex');

kernel = kernel / sum(kernel(:));

kernel_fft = fftshift(fft2(kernel));
img_d_noisy_fft = fftshift(fft2(img_d_noisy));

filtered_fft = kernel_fft .* img_d_noisy_fft;
img_filtered = fftshift(abs(ifft2(filtered_fft)));
img_filtered = img_filtered / max(img_filtered(:));

figure('units','normalized','outerposition',[0 0 1 1]);

subplot(2,2,1);
imshow(img_d);
title('Original Image - First Slice', 'Interpreter', 'latex');

subplot(2,2,2);
imshow(img_d_noisy);
title('Image with Added Gaussian Noise', 'Interpreter', 'latex');

subplot(2,2,3);
imshow(img_filtered);
title('Filtered Using Rectangular Kernel', 'Interpreter', 'latex');

img_filtered_2 = imgaussfilt(img_d, 1);
subplot(2,2,4);
imshow(img_filtered_2);
title('Filtered Using Gaussian Kernel on Original Image', 'Interpreter', 'latex');
```

the output of the code is shown in below figures:

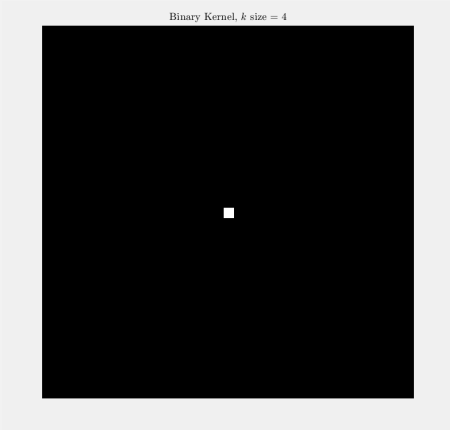


Figure 1. binary kernel

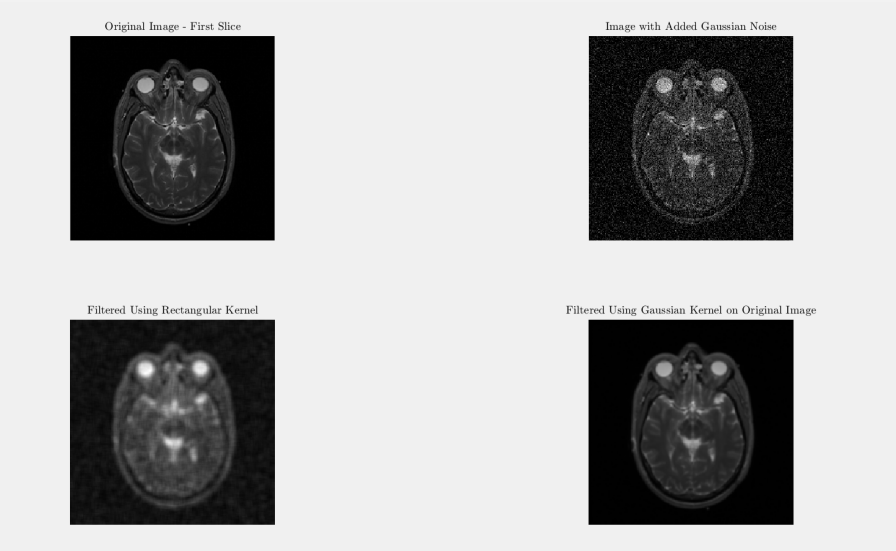


Figure 2. output of the code

After comparing the outcomes of the two filtering techniques, it is found that the square kernels smooth the image by averaging close pixels, which effectively eliminates salt-and-pepper noise but also causes some edge blurring. However, the Gaussian filter on the other hand, more efficiently reduces high-frequency components while offering smoother edge preservation and improved noise reduction overall.

Question 2

Read the first slice of the image `t2.jpg` from the `S2_Q2_utils` directory. Blur the image significantly by applying a Gaussian filter (h) with a large variance, resulting in a blurred image. Then, reconstruct the original image using the formula

$$F = \frac{G}{H}.$$

Compare the reconstruction with the original and analyze the results.

Add Gaussian noise with a variance of 0.001 to the blurred image, apply the convolution filter, and attempt reconstruction again. Analyze and compare the outcomes.

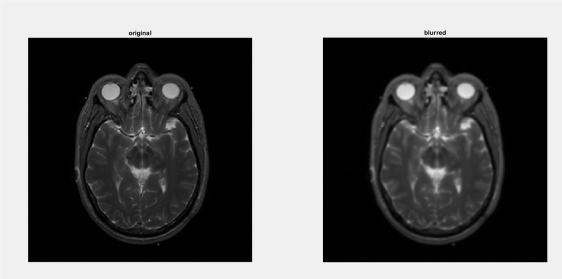
solution

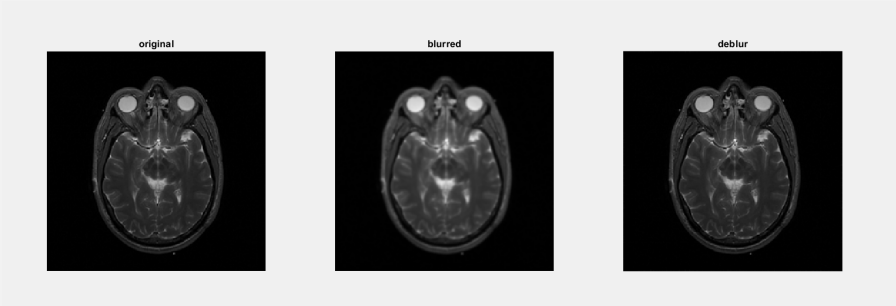
we used the below code for this part:

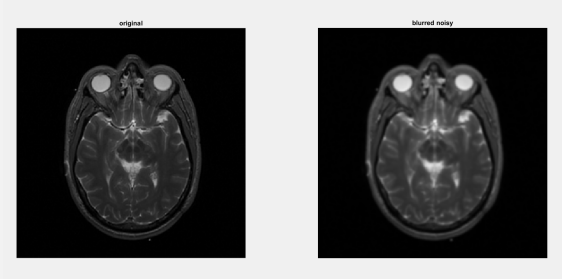
```
clc;clear;close all;
image = imread("S2_Q2_utils/t2.jpg");
image=image(:,:,1);
f = double(image);
h = Gaussian(1, [256 256]);
g=conv2(f,h,'same');
figure()
subplot(1,2,1)
imshow(f/255)
title('original')
subplot(1,2,2)
imshow(g/max(g,[],'all'))
title('blurred')

G_2=fftshift(fft2(ifftshift((g))));
H=fftshift(fft2(ifftshift((h))));
F=G_2./H;
f_hat=fftshift(ifft2(ifftshift(F)));
figure()
subplot(1,3,1)
imshow(f/255)
title('original')

subplot(1,3,2)
imshow(g/max(g,[],'all'))
title('blurred')
subplot(1,3,3)
imshow(abs(f_hat)/max(f_hat,[],'all'));
title('deblur')
g_noise=g+randn(size(g))*0.001;
figure()
subplot(1,2,1)
imshow(f/255)
```

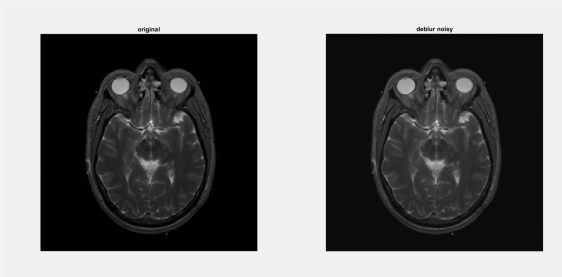
```
title('original')
subplot(1,2,2)
imshow(g_noise/max(g_noise,[],'all'))
title('blurred noisy')
G_2=fftshift(fft2(ifftshift((g_noise))));
H=fftshift(fft2(ifftshift((h))));
F=G_2./H;
f_hat=fftshift(ifft2(ifftshift(F)));
f_hat=f_hat-min(f_hat,[],'all');
figure()
subplot(1,2,1)
imshow(f/255)
title('original')

subplot(1,2,2)
imshow(f_hat/max(f_hat,[],'all'));
title('deblur noisy')
```

the outputs are shown in below figures.









The deblurring process involves dividing the Fourier transform of the blurred image G by the Gaussian filter H, and reconstructing the image using the inverse Fourier transform. For the noise-free case, the deblurred image recovers much of the original structure and edges, appearing sharper than the blurred image, though minor artifacts may persist depending on the accuracy of the filter h.
When Gaussian noise with a small variance (0.001) is added to the blurred image, it becomes grainy and less smooth. Applying the same deblurring method amplifies the noise due to the division in the frequency domain, introducing noticeable artifacts in the reconstructed image. While edges are sharpened, the reconstruction is degraded compared to the noise-free case, resulting in less accurate restoration.

Question 3

In this question, we aim to compute the matrix $D \in \mathbb{R}^{N^2 \times N^2}$, knowing the blurring filter h, and use it to reconstruct the initial image from the previous question using the equation $f = D^{\dagger} g$. To reduce computational overhead, first reduce the size of the initial image to 64×64. Note that each row of the matrix D corresponds to implementing the filtering effect for one pixel. Start by creating an all-zero image K with the same dimensions as the initial image, and place the filter h in the upper-left corner of this image. Then, for the pixel at location r, c, take the corresponding row in the matrix D by first shifting the image K by $c-1$ columns and $r-1$ rows. Explain why the entire rows of D correspond to a Toeplitz structure.

Construct the blurring matrix D and simulate the effect of the blurring filter h on the initial image (e.g.,

$$h = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix} \text{ or a Gaussian filter).}$$

Then, add Gaussian noise with a standard deviation of 0.05 to Df to obtain the noisy blurred image g. Finally, reconstruct the initial image using the equation $f = D^{\dagger} g$ and analyze the results by displaying the initial image, the blurred image, and the reconstructed image, discussing the differences.

solution

we used this code for this part:

```
clc; clear; close all;

image = imread("S2_Q2_utils/t2.jpg");
img_resized = imresize(image(:,:,1), [64, 64], 'bilinear');
f = double(img_resized);

h = [0 1 0; 1 2 1; 0 1 0];
K = zeros(64);
D = zeros(64*64);

K(1:3, 1:3) = h;
for count = 1:64*64
    [r, c] = ind2sub([64, 64], count);
    temp = circshift(K, [r-1, c-1]);
    D(count, :) = temp(:)';
end

figure();
spy(D);

g = D * f(:);
g_reshape = reshape(g, 64, 64);

figure();
subplot(1, 3, 1);
imshow(g_reshape, []);
title('Blurred', 'Interpreter', 'latex');

g_noise = g_reshape + randn(size(g_reshape)) * 0.05;

subplot(1, 3, 2);
imshow(g_noise, []);
title('Noisy', 'Interpreter', 'latex');

f_hat = pinv(D) * g_noise(:);
f_reconstructed = reshape(f_hat, 64, 64);

subplot(1, 3, 3);
imshow(f_reconstructed, []);
title('Reconstructed', 'Interpreter', 'latex');
```

the output of the code is shown in below figures:

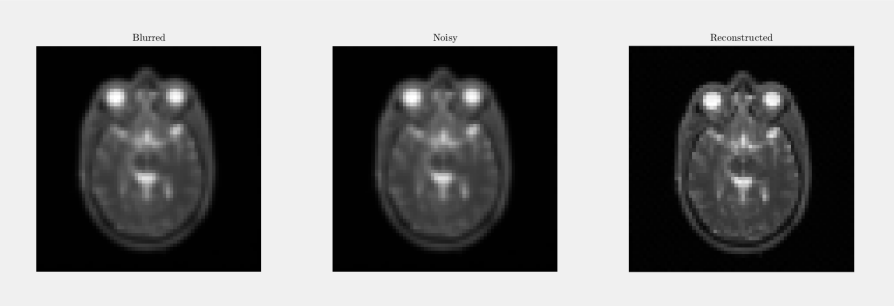


Figure 3. the output of reconstruction

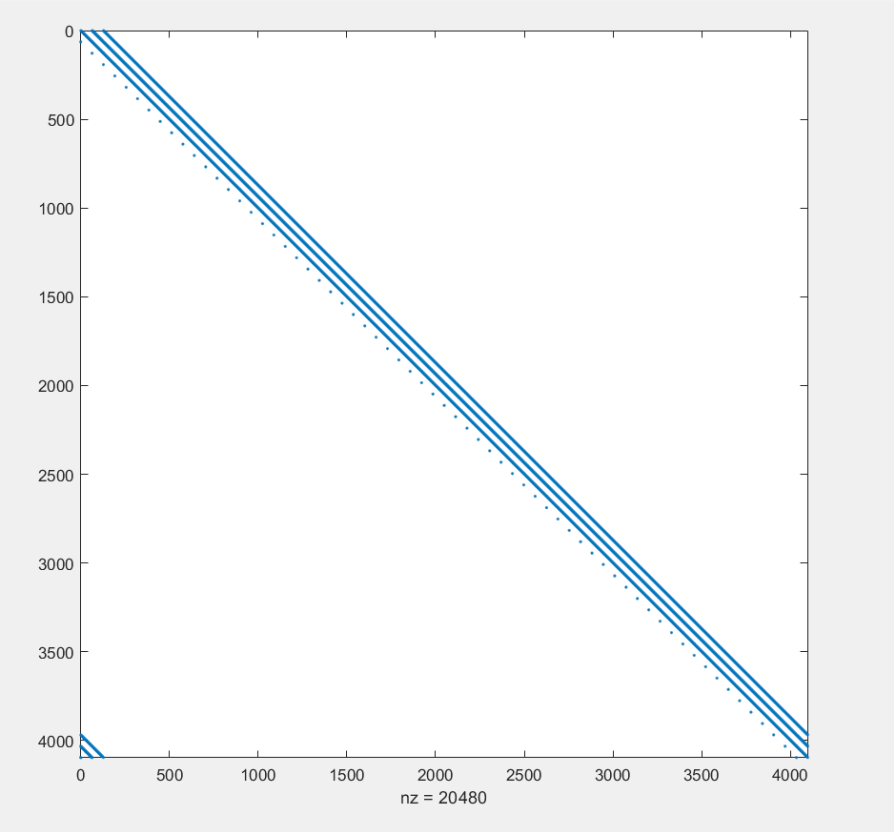


Figure 4. D matrix

The matrix D represents the Toeplitz structure created by applying the blurring filter h across the image. Each row of D corresponds to the shifted version of the filter h applied at a specific pixel location. Its sparse structure, as visualized with `spy(D)`, indicates that only the neighboring pixels affected by the filter contribute to each row. This results in a banded diagonal pattern.
The comparison between blurred, noisy, and reconstructed images shows that the blurring removes fine details, and adding noise further degrades the image quality. Using the pseudo-inverse $D^{\dagger}$ for reconstruction restores some structures, but the noise introduces artifacts and reduces the quality compared to the original.

Question 4

Implement the Gradient Descent algorithm and execute the steps from the previous question, comparing the results. Additionally, analyze the convergence of different iterations of the algorithm through simulation. In your opinion, why does the Gradient Descent method often perform better than the method used in the previous question?

solution

for this part we used this code:

```
beta = 0.01;
f_k = zeros(64*64, 1);

for i = 1:50
    f_k = f_k + beta * D' * (g - D * f_k);
end

figure;
imshow(reshape(f_k, 64, 64), []);
title('Reconstructed with Gradient', 'Interpreter', 'latex');
```

the output is shown in below figure:

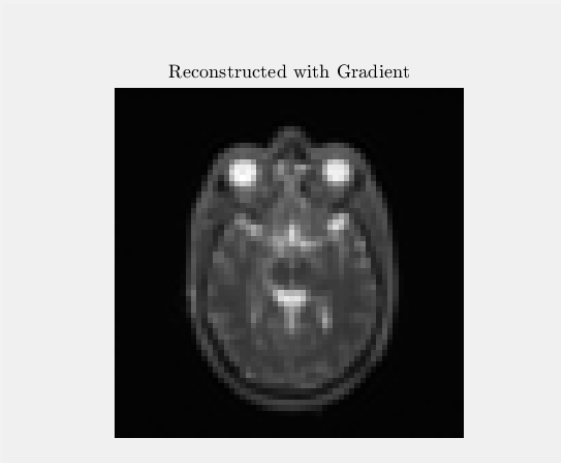


Figure 5. output of gradient decsent

Gradient Descent is often better than the previous method (pseudo-inverse $D^{\dagger}$) because it iteratively minimizes the error and avoids directly computing the inverse of the matrix D, which can be computationally expensive and numerically unstable, especially for large and ill-conditioned matrices. The pseudo-inverse amplifies noise due to its sensitivity to small singular values, while Gradient Descent can mitigate this issue by controlling the learning rate and iteratively refining the solution. This iterative approach also allows for early stopping, reducing noise amplification and improving convergence behavior in noisy scenarios.

Question 5

The methods mentioned for noise removal and interference reduction from medical images are among the most advanced techniques in this field. More advanced methods, such as the Anisotropic Diffusion Filtering method, have also been proposed. Research this method and its implementation. Additionally, execute the related code (`S2_Q5_Anisotropic_Diffusion.m`) and analyze the process and its execution.

solution

The method reduces noise over time by iteratively solving a diffusion-based Partial Differential Equation (PDE). The result is a smoothed image where edges are preserved, which is critical for medical imaging applications. Compared to uniform blurring, anisotropic diffusion achieves a better balance between noise reduction and edge preservation.

By adjusting parameters such as the edge threshold K, time duration t_{end}, and diffusion scaling factor λ, the performance of the method can be fine-tuned. Lower K values preserve finer details, while longer diffusion times enhance noise removal but risk over-smoothing.

In comparison to simpler methods like uniform blurring, anisotropic diffusion demonstrates superior performance. It avoids excessive edge loss and offers flexibility to handle images with varying noise levels and structural complexities. The iterative nature of the approach ensures effective smoothing while maintaining critical edge information.